

雲原生 HW4 - CI/CD Pipeline

資管三 B12705041 李健安

May 16, 2026

My GitHub Repository: <https://github.com/LEE-CHIEN-AN/cicd-lab>

CI Pipeline 說明

.github/workflows/ci_b12705041.yaml 如下

```
1 name: CI b12705041
2
3 on:
4   push:
5     branches:
6       - "*" # 所有 branch push 都觸發
7   pull_request: # PR 也觸發，保護 main branch
8
9 permissions:
10  contents: read # 讀取 repo 內容，actions/checkout 需要這個權限
11  checks: write # dorny/test-reporter 需要這個才能寫入 GitHub Checks 系統，才能讓測試通過/失敗的圖示出現在 Actions 頁面
12
13 jobs:
14   ci:
15     runs-on: ubuntu-latest # GitHub 幫你開一台全新的 Ubuntu 虛擬機來跑這些步驟。
16
17     steps:
18       - name: Checkout # 把你 repo 的程式碼下載進 VM。幾乎每個 workflow 的第一步都是這個，沒有 code 後面什麼都跑不了。
19         uses: actions/checkout@v5
20
21       - name: Setup Node.js
22         uses: actions/setup-node@v5
23         with:
```

```

24     node-version: "22"
25     cache: npm #優化：把 node_modules 的下載結果快取起來。下
        次跑 CI 時如果 package-lock.json 沒變，就直接用快取
26
27     - name: Install dependencies
28       run: npm ci
29
30     - name: TypeScript typecheck
31       run: npm run typecheck #實際執行的是 tsc --noEmit，意思是「用
        TypeScript 編譯器檢查型別，但不要輸出任何檔案」。
32
33     - name: Prettier check #實際執行 prettier --check .，檢查所有程
        式碼的格式是否符合 .prettierrc 的規定（縮排、引號、分號等）
34       run: npm run format:check #這個步驟只「檢查」，不會自動修正。
        格式不對就直接 fail。
35
36     - name: Lint
37       run: npm run lint
38
39     - name: Run tests #--reporter=default：印在 log 裡，讓你在
        Actions 頁面的 log 看到測試結果，--reporter=junit：輸出
        vitest-junit.xml，這是業界標準的 XML 格式，給下一步的 test-
        reporter 用
40       run: |
41         mkdir -p reports
42         npm test -- --reporter=default --reporter=junit --
            outputFile.junit=reports/vitest-junit.xml
43
44     - name: Publish test results
45       uses: dorny/test-reporter@v1
46       if: always() # 不管測試成功或失敗都要執行這個步驟，這樣才能在
        GitHub Actions 頁面看到測試結果
47       with:
48         name: Vitest Test Results
49         path: reports/vitest-junit.xml
50         reporter: java-junit

```

CI Pipeline 的流程如下：push 觸發 ↓ 開一台乾淨的 Ubuntu VM ↓ 下載 code (Checkout)
 ↓ 安裝 Node 22 (Setup Node.js) ↓ 安裝套件 (npm ci) ↓ 型別檢查 (typecheck) → 失敗就停
 ↓ 格式檢查 (prettier check) → 失敗就停 ↓ 跑測試 (vitest) → 失敗會繼續跑下一步 ↓ 發布測
 試報告 (dorny/test-reporter) ← if: always() 確保一定跑 ↓ VM 刪除

CI 執行結果截圖

- 至少一張成功執行的截圖 - 附上 GitHub Actions 頁面的結果畫面

失敗案例說明

- 請故意製造一個錯誤，例如：
 - TypeScript 型別錯誤
 - Prettier 格式錯誤
 - 測試失敗
- 附上 pipeline failed 的截圖, 說明錯誤原因與修正方式